

Internship Report — Day 9

Internship Role: Cloud / Infrastructure Deployment

Technology Focus: Azure Resource Manager (ARM) Templates

Date: Day 9 of Internship

Objective of the Day

The objective of Day 9 was to gain hands-on experience with Azure Resource Manager (ARM) templates by observing, understanding, and deploying infrastructure through automation. The focus was on learning the deployment workflow, resource dependencies, and template structure.

Task 1 — Deploy ARM Template and Observe Resource Creation Flow

I was assigned an ARM template deployment that provisions the following resources:

- Storage Account
- Virtual Machine
- File Share

Key Learning Goals

- Understand the deployment procedure
- Observe the order in which resources are created
- Identify resource dependencies

Observations

1. **Resource Group Validation**
 - Deployment begins by validating the template and parameters.
 - Resource group acts as the container for all resources.
2. **Storage Account Deployment**
 - Storage account is created first because it may be required by other services (e.g., diagnostics, file share).
3. **File Share Creation**
 - File share is created inside the storage account.
 - This confirms dependency on the storage account.

4. Virtual Machine Deployment

- VM provisioning happens after networking and storage components are prepared.
- VM depends on:
 - Virtual network
 - Network interface
 - Public IP
 - Storage for OS disk or boot diagnostics

Resource Dependency Understanding

Resource	Depends On
File Share	Storage Account
Network Interface	Virtual Network + Subnet
Virtual Machine	Network Interface + Storage
Boot Diagnostics	Storage Account

Deployment Insight

ARM templates automatically manage dependency order using the `dependsOn` property or implicit references.

Task 2 — Study ARM Template Structure

The second task involved reading and analyzing the ARM template in detail to understand its structure and components.

ARM Template Core Structure

1. **Schema**
 - Defines template format version.
2. **Parameters**
 - Inputs provided during deployment.
 - Example: VM name, OS type, admin credentials.
3. **Variables**
 - Derived values to avoid repetition.
 - Used for naming conventions and configurations.
4. **Resources**
 - Actual infrastructure components to deploy.
5. **Outputs**
 - Values returned after deployment (e.g., public IP).

Key Understanding

- Templates follow a JSON format.
 - Parameters make templates reusable.
 - Variables simplify configuration logic.
 - Resource blocks define infrastructure.
 - Outputs provide deployment results.
-

Task 3 — Conditional OS Deployment with Web Server Installation

The third task was to deploy a virtual machine using an ARM template with **conditional OS selection**.

Deployment Requirements

The template should allow switching between:

Option 1 — Linux VM

- OS: Linux
- Web Server: Nginx
- Custom HTML page deployed via script
- Access via Public IP

Option 2 — Windows VM

- OS: Windows
- Web Server: IIS
- Custom HTML page deployed
- Access via Public IP

Implementation Concept

- Boolean or parameter-based OS selection.
- Conditional resource configuration.
- Custom script extension used for software installation.

Verification Process

After deployment:

1. Retrieve VM public IP.
2. Open browser.
3. Access hosted HTML page.
4. Confirm web server is running correctly.

Result

- Linux deployment successfully hosted webpage using Nginx.
 - Windows deployment successfully hosted webpage using IIS.
 - Conditional logic in ARM template worked correctly.
-

Skills and Knowledge Gained

- Understanding infrastructure as code (IaC).
 - Learning ARM template architecture.
 - Identifying resource dependencies.
 - Using conditional deployment logic.
 - Automating software installation with custom scripts.
 - Validating web service accessibility via public IP.
-

Conclusion

Day 9 provided practical exposure to automated infrastructure deployment using ARM templates. I gained hands-on understanding of how Azure provisions resources, manages dependencies, and enables flexible deployments through parameterization and conditional logic. This experience strengthened my knowledge of Infrastructure as Code and cloud automation practices.
